BASH SCRIPTING

Week 4
Loops and Repetition

Repeat safely, count outcomes, skip, and stop

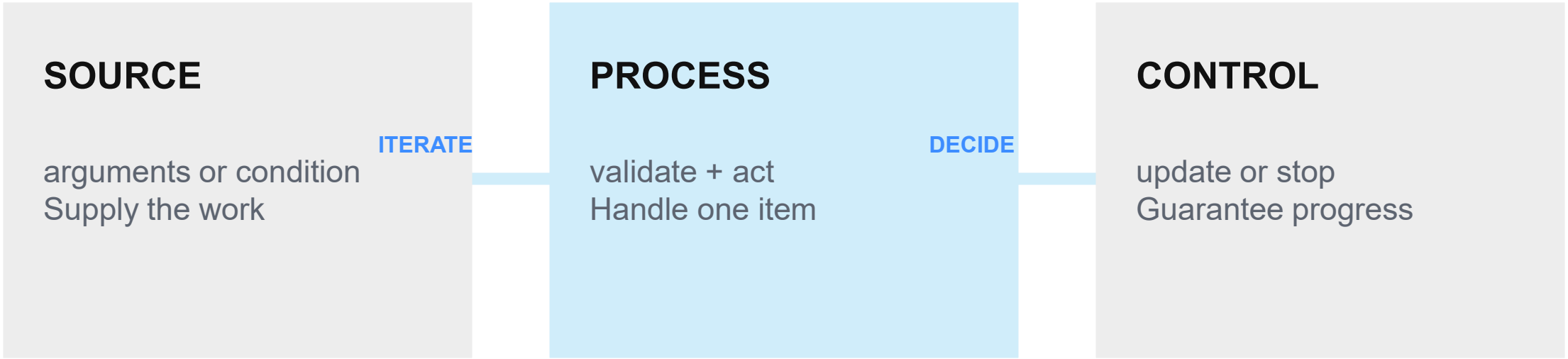
Two 90-minute lessons | Milestone M4: Batch Processor

Instructor: Muhammad Khalid Khan

By the end of Week 4, you can control repeated work

01	Explain initialize, test, process, update, and stop
02	Iterate safely over quoted positional arguments
03	Choose for, while, or until for the task
04	Maintain accurate processing counters
05	Skip recoverable items with continue
06	Stop intentionally with break and report a summary

Reliable loops preserve data and guarantee a stop



A loop is safe when every iteration preserves data and moves toward termination.

Loop control drives batch processing

1

INITIALIZE

Set counters

2

TEST

More work?

3

PROCESS

Handle item

4

UPDATE

Repeat or stop

```
initialize -> test -> process -> update -> repeat or stop -> summary
```

Use for when the collection is already known

```
total=0
for item in "$@"; do
  ((total += 1))
  printf 'Item %s: %s\n' "$total"
"$item"
done

printf 'Total: %s\n' "$total"
```

01 **SOURCE**

Quoted "$@" preserves arguments.

02 **ITERATION**

One argument becomes one item.

03 **COUNTER**

Initialize before the loop.

04 **SUMMARY**

Report after the loop finishes.

Match the loop to the repetition source

KNOWN COLLECTION

```
for item in "$@"
```

Use for arguments, globs, or a numeric sequence.
The item source is known.

CONDITION CONTROL

```
while condition
```

Use while or until when a condition controls repetition.

The clearest loop makes its source and stopping rule visible.

DEMO

Guided demo: preserve and count every argument

```
#!/usr/bin/env bash

total=0
processed=0
skipped=0

for item in "$@"; do
  ((total += 1))
  if [[ -z "$item" ]]; then
    ((skipped += 1))
    continue
  fi
  ((processed += 1))
  printf '%s: <%s>\n' "$processed" "$item"
done
```

TEST

```
./argument-batch-
reporter.sh

One item
A value with spaces
An empty argument
```

CHECK
Quoted items and counters must agree with the input.

continue skips; break stops the nearest loop

CONTINUE

Skip current item

```
missing -> count ->
continue
```

BREAK

Leave the loop

```
stop mode -> break
```

SUMMARY

Runs after the loop

```
print final counters
```

Both controls need an explicit, testable reason.

Condition loops must change state or observe progress

WHILE

`while condition`

Repeat while the test succeeds
and update relevant state

UNTIL

`until condition`

Repeat until the test succeeds
and enforce a maximum

```
attempt=1; until (( attempt > maximum )); do ...; ((attempt += 1)); done
```

DEMO

A batch processor skips errors or stops early

```
for path in "$@"; do
  ((total += 1))
  if [[ ! -e "$path" ]]; then
    ((missing += 1))
    if (( stop_on_error == 1 )); then
      stopped_early=1
      break
    fi
    continue
  fi
  process_path "$path"
done

print_summary
```

CONTROL ORDER

01 Increment attempted count

02 Validate current item

03 Count the failure

04 Continue or break

05 Print final summary

Critical test: counters must reconcile after both skip and stop paths.

Two labs turn repetition into batch evidence

LAB 1

Argument Batch Reporter

Iterate over quoted "$@"
Number each item
Skip empty arguments
Maintain three counters
Verify zero and many inputs

LAB 2

Safe Batch Processor

Classify multiple paths
Continue after missing paths
Support stop-on-error
Break intentionally
Reconcile the summary

Required proof: zero + one + many + spaces + continue + break + final counters

Milestone M4: repeat, count, skip, stop, and report

BATCH PROCESSOR

You are ready when every item stays intact, counters reconcile, and the loop always terminates.

01 **ITERATE**
Correct loop + safe item source

02 **CONTROL**
Accurate counters + continue or break

03 **REPORT**
Summary + documented exit status

NEXT: Week 5 adds functions, parameters, return status, and reusable design.